ECOLE NATIONALE DES SCIENCES APPLIQUEES
AL HOCEIMA

Annee Universitaire 2025-2026

Module : Applications Web Avancees

avec Java et Spring

COMPTE RENDU DE PROJET

Transformation MVC 2 avec Hibernate

Amelioration d'une application existante vers le patron d'architecture MVC 2, en utilisant Maven pour la gestion du projet et l'ORM Hibernate pour la persistance des donnees sous MySQL.

Realise par :

Ayoub Nassih

Encadre par :

M. Cherradi

20/04/2026

0 1

1 Introduction **2**
- 1.1 Architecture du projet dans Eclipse 3
- 1.2 Structure des fichiers 4

2 Interfaces graphiques **4**
- 2.1 Page d'authentification 5
- 2.2 Page d'inscription 6
- 2.3 Liste des produits 6
- 2.4 Ajout d'un produit 7
- 2.5 Recherche d'un produit 7

3 Architecture Detaillee MVC 2 **7**
- 3.1 Schema du flux de requetes 7

4 Diagramme de Sequence **8**

5 Configuration du projet (Maven) **8**

6 Couche Persistance avec Hibernate **9**

7 Couche DAO **10**

8 Couches Controleur et Vue **11**

9 Validation et Tests **12**

10 Conclusion **12**

1 1

Objectifs du Projet

- Structurer une application Java web selon le patron d'architecture **MVC 2**.
- Gerer le cycle de vie du projet et ses dependances via **Maven**.
- Implementer la couche de persistance des donnees (MySQL) avec l'ORM **Hibernate**.
- Securiser l'acces et enrichir l'interface graphique (Bootstrap & JSTL).

Le present rapport detaille la restructuration de l'application de *Gestion de Produits*. Cette evolution majeure permet de passer d'un modele de script Java statique a une application Web complete.

1.1 1Architecture du projet dans Eclipse

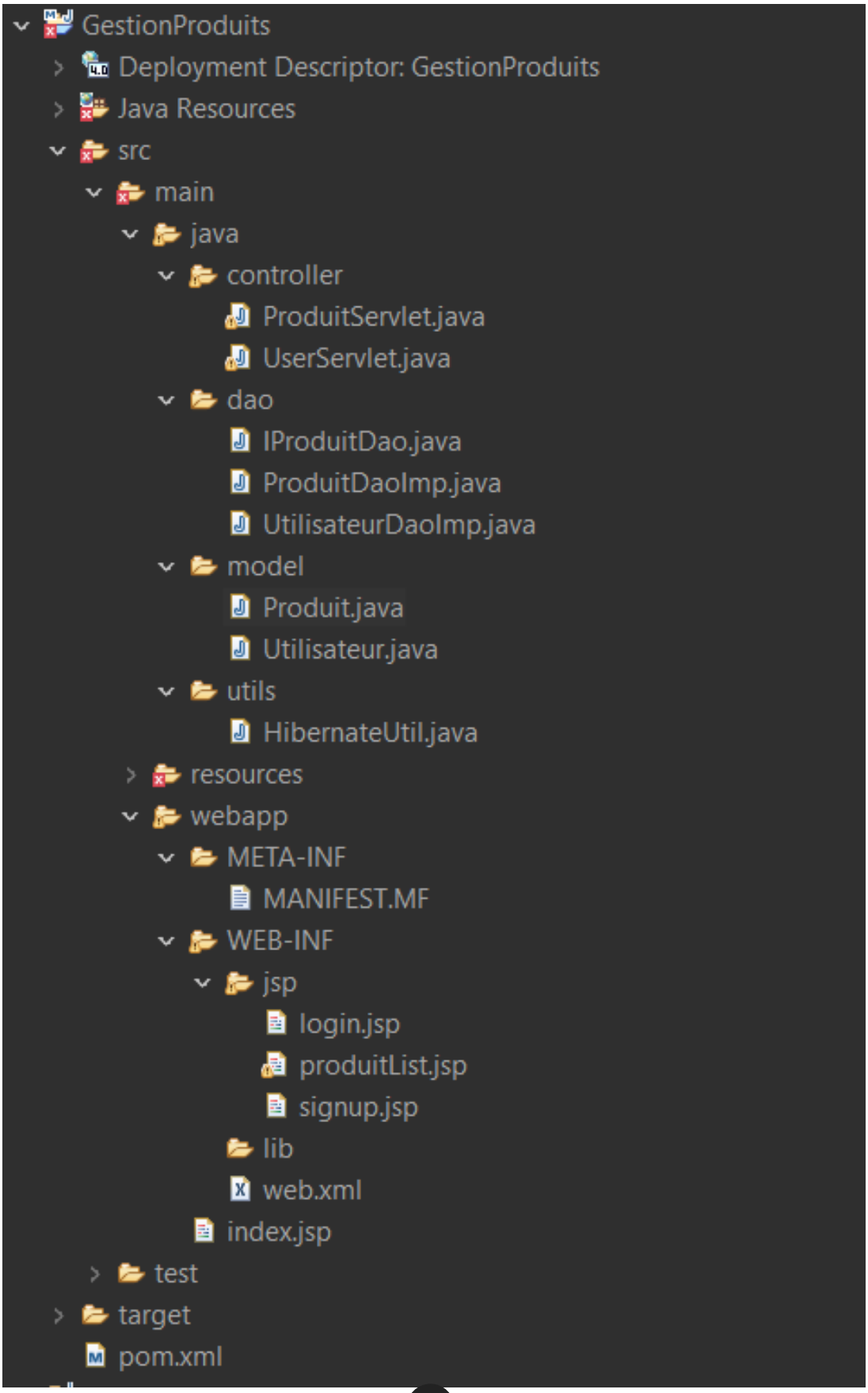


FIGURE 1 – Architecture du projet dans Eclipse

1.2 1Structure des fichiers

Arborescence du projet | text

```text
GestionProduits/
|-- pom.xml
|-- src/main/java/
|   |-- controller/
|   |   |-- ProduitServlet.java
|   |   `-- UserServlet.java
|   |-- dao/
|   |   |-- IProduitDao.java
|   |   |-- ProduitDaoImp.java
|   |   `-- UtilisateurDaoImp.java
|   |-- model/
|   |   |-- Produit.java
|   |   `-- Utilisateur.java
|   `-- utils/
|       `-- HibernateUtil.java
|-- src/main/resources/
|   |-- hibernate.cfg.xml
|   |-- Produit.hbm.xml
|   `-- Utilisateur.hbm.xml
`-- src/main/webapp/
    `-- WEB-INF/
        |-- web.xml
        `-- jsp/
            |-- login.jsp
            |-- produitList.jsp
            `-- signup.jsp
```

2 1

2.1 1Page d'authentification

FIGURE 2 – Formulaire de connexion

2.2 1Page d'inscription

FIGURE 3 – Formulaire d'inscription

2.3 1Liste des produits

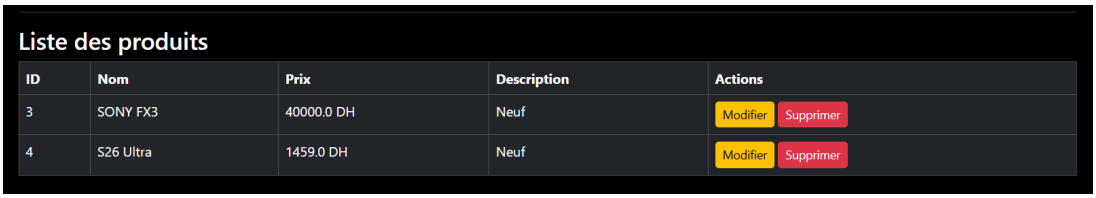


FIGURE 4 – Liste des produits avec operations CRUD

2.4 1Ajout d'un produit

FIGURE 5 – Formulaire d'ajout de produit

2.5 1Recherche d'un produit

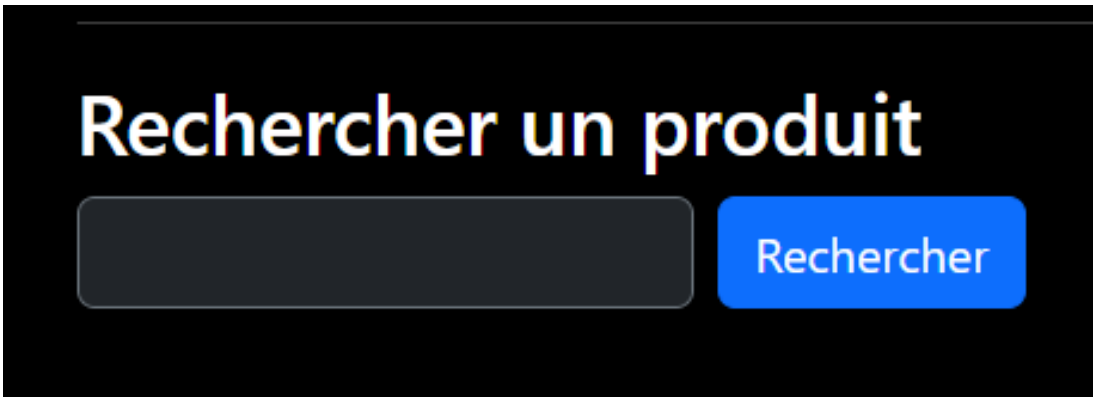
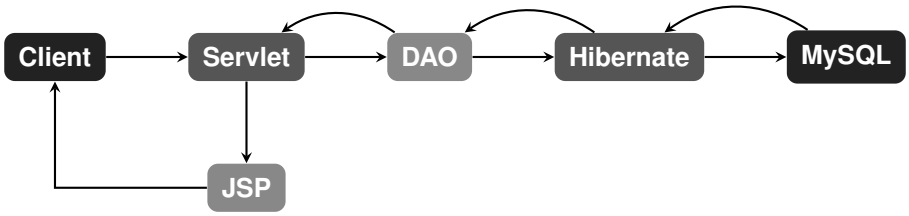


FIGURE 6 – Recherche de produit par ID

3 1

3.1 1Schema du flux de requetes



4 1

Etape	Acteur / Composant	Action
1	Utilisateur	Remplit le formulaire et le soumet
2	Servlet doPost	Recoit la requete add
3	DAO.ajouter()	Debute la transaction
4	Session.save()	Insertion MySQL sous Hibernate
5	Transaction.commit()	Validation de l'ajout
6	Servlet	Redirection vers la liste des produits

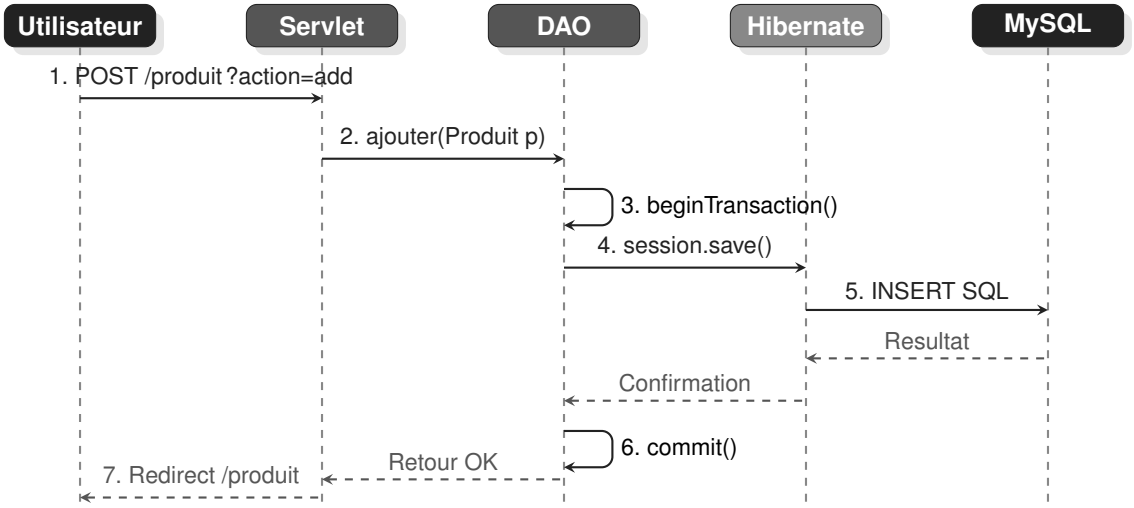


FIGURE 7 – Diagramme de sequence - Ajout produit

5 1

Categorie	Technologie	Version	Role
Build	Maven	3.x	Gestion de dependances
ORM	Hibernate	5.6.15	Mapping objet-relationnel
Base de donnees	MySQL	8.x	Stockage
Serveur	Apache Tomcat	9.0/10.1	Conteneur web

6 1

Configuration Hibernate | XML

```xml
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/MVC2</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <mapping resource="Produit.hbm.xml"/>
        <mapping resource="Utilisateur.hbm.xml"/>
    </session-factory>
</hibernate-configuration>
```

HibernateUtil Singleton | Java

```java
public class HibernateUtil {
    private static final SessionFactory sessionFactory;
    static {
        try {
            Configuration config = new Configuration();
            config.configure();
            sessionFactory = config.buildSessionFactory();
        } catch (Throwable ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }
    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

7 1

Interface IProduitDao | Java

```java
public interface IProduitDao {
    void ajouter(Produit p);
    void supprimer(Long id);
    void modifier(Produit p);
    Produit trouverParId(Long id);
    List<Produit> listerTous();
}
```

Implementation DAO avec Hibernate | Java

```java
public class ProduitDaoImp implements IProduitDao {
    @Override
    public void ajouter(Produit p) {
        Transaction transaction = null;
        try(Session session = HibernateUtil.getSessionFactory().
            openSession()){
            transaction = session.beginTransaction();
            session.save(p);
            transaction.commit();
        } catch(Exception e) {
            if(transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }
}
```

8 1

URL	Methode	Action	Redirection
`/produit?action=list`	GET	Afficher tous les produits	Forward JSP
`/produit?action=add`	POST	Ajouter produit	Redirect /produit
`/user?action=login`	POST	Connexion	Redirect /produit

TABLE 1 – Tableau des routes du controleur

Extrait JSP avec JSTL | HTML

```
<%-- Verif Session --%>
<% if (session.getAttribute("user") == null) {
    response.sendRedirect("user?action=login");
    return;
} %>

<c:forEach items="${produits}" var="p">
    <tr>
        <td>${p.id}</td>
        <td>${p.nom}</td>
        <td>${p.prix} DH</td>
    </tr>
</c:forEach>
```

9 1

ID	Fonctionnalite	Resultat attendu	Statut
T01	Inscription	Compte cree en base	PASS
T02	Connexion valide	Session creee, acces dashboard	PASS
T03	Ajout produit	Produit liste apres redirection	PASS
T04	Recherche produit	Produit affiche par ID	PASS

TABLE 2 – Tests de validation

10 1

Ce projet valide l'acquisition des competences sur l'architecture standard MVC 2 avec Java EE. L'utilisation de **Hibernate** avec **MySQL** demontre la puissance des objets manages, s'affranchissant totalement de la redaction manuelle des requetes SQL.